

Convex Boundary Avoidance

Kaleb Ben Naveed

Algorithm 1 The Convex Boundary avoidance algorithm

```

1: function CvxBoundCorrection(CvxShape,  $x$ ,  $u$ ;  $d_{min} = 1.0$ ,  $s_{max} = 1.8$ )
2:    $\triangleright$  state vector  $x \in \mathbb{R}^2$  and control input  $u \in \mathbb{R}^2$ 
3:    $\triangleright$  CvxShape struct contains information about the
     vertices of the defined convex domain
4:    $\triangleright d_{min} \in \mathbb{R}$  is the minimum safe distance
5:    $\triangleright s_{max} \in \mathbb{R}$  is the maximum speed of the robot
6:
7:    $d, p_{clo} = \text{MinLenToBoundary}(\text{CvxShape}, x)$ 
8:  $\triangleright d \in \mathbb{R}$  is the distance to the closet point  $p_{clo} \in \mathbb{R}^2$  on
   the boundary
9:    $p_{cen} = \text{CvxShapeCentroid}(\text{CvxShape})$ 
10:   $\triangleright p_{cen} \in \mathbb{R}^2$  is the center point of the convex shape
11:   $n_p = \text{NormalVecCloCen}(p_{cen}, p_{clo})$ 
12:   $\triangleright n_p \in \mathbb{R}^2$  is the normal vector from the closet point
      $p_{clo}$  on the boundary to the centroid  $p_{cen}$ 
13:   $u_{ff} = s_{max} n_p$ 
14:   $\triangleright u_{ff} \in \mathbb{R}^2$  is the force field vector
15:   $u_{cmd} = \min(1, \frac{d}{d_{min}})u + \max(0, 1 - \frac{d}{d_{min}})u_{ff}$ 
16:  return  $u_{cmd}$ 
17: end function

```

Algorithm 2 Minimum length to Boundary

```

1: function MinLenToBoundary(CvxShape,  $x$ )
2:    $n_{edges} = \text{size}(\text{CvxShape.vertices}, 2)$ 
3:    $d = \infty$ 
4:   for  $i \in 1 : n_{edges}$  do
5:      $v_1 = \text{CvxShape.vertices}[:, i]$ 
6:      $v_2 = \text{CvxShape.vertices}[:, \text{mod}(i, n_{edges}) + 1]$ 
7:      $d_{temp}, p_{clo, temp} = \text{PointProjLine}(x, v_1, v_2)$ 
8:     if  $d_{temp} < d$  then
9:        $d = d_{temp}$ 
10:       $p_{clo} = p_{clo, temp}$ 
11:    end if
12:  end for
13:  return  $d, p_{clo}$ 
14: end function

```

Algorithm 3 Projection of Point on Line

```

1: function PointProjLine( $x, v_1, v_2$ )
2:    $v_{vec} = v_2 - v_1$ 
3:    $p_{vec} = x - v_1$ 
4:    $\hat{v} = \frac{v_{vec}}{\|v_{vec}\|}$ 
5:    $l_{proj} = p_{vec} \cdot \hat{v}$   $\triangleright$  Dot product to compute
     projection length
6:   if  $l_{proj} < 0$  then
7:      $d_{temp} = \|x - v_1\|$ 
8:      $p_{clo, temp} = v_1$ 
9:   else if  $l_{proj} > 0$  then
10:     $d_{temp} = \|x - v_2\|$ 
11:     $p_{clo, temp} = v_2$ 
12:   else
13:     $p_{proj} = v_1 + l_{proj} \hat{v}$ 
14:     $d_{temp} = \|x - p_{proj}\|$ 
15:     $p_{clo, temp} = p_{proj}$ 
16:   end if
17:   return  $d_{temp}, p_{clo, temp}$ 
18: end function

```
